Introduction

This unit discusses dynamic memory allocation in the C programming language is a crucial aspect of managing memory resources during program execution. Unlike static memory allocation, which occurs at compile time, dynamic memory allocation allows for the allocation and deallocation of memory at runtime, offering flexibility in memory usage.

Much of the power of pointers stems from their ability to track dynamically allocated memory. The management of this memory through pointers forms the basis for many operations, including those used to manipulate complex data structures.

Runtime System

Stack and Heap

C program executes within a runtime system. This is typically the environment provided by an operating system.

The runtime system supports the stack and heap along with other program behavior.

Memory Management

Memory management is central to all programs, sometimes managed implicitly by the runtime system.

Instead of having to allocate memory to accommodate the largest possible size for a data structure, only the actual amount required needs to be allocated.

💾 Dynamic Memory Allocation

The Creating and maintaining dynamic structures requires dynamic memory allocation the ability for a program to obtain more memory space at execution time to hold new values, and to

release space no longer needed.

While doing programming, if you are aware about the size of an array, then it is easy and you can define it as an array.

Example:

To store a name of any person, it can go max 100 characters as follows:

```
char name[100]
```

But now let us consider a situation where you have no idea about the length of the text you need to store, for example you want to store a detailed description about a topic. Here we need to define a pointer to character without defining how much memory is required and later based on requirement we can allocate memory.

Code Example:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
  char name[100];
  char *description;
  strcpy(name, "Raghav Rai");
  /* allocate memory dynamically */
  description = malloc( 200 * sizeof(char) );
  if( description == NULL )
  {
    fprintf(stderr, "Error - unable to allocate required memory\n");
  }
  else
  {
    strcpy( description, "Raghav Rai a MVM student in class 11th");
  }
  printf("Name = %s\n", name );
  printf("Description: %s\n", description );
}
```

Output:

```
Name = Raghav Rai
Description: Raghav Rai a MVM student in class 11th
```

In case this program can be written using calloc(); only thing is you need to replace malloc with calloc as follows:

```
calloc(200, sizeof(char));
```

The complete control and pass any size value while allocating memory, unlike arrays where once the size is defined, then after you cannot change it.

Resizing and Releasing Memory

When your program comes out, operating system automatically release all the memory allocated by your program but as a good practice when you are not in need of memory

anymore then you should release that memory by calling the function free().

Alternatively, you can increase or decrease the size of an allocated memory block by calling the function realloc().

Example with realloc() and free():

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
  char name[100];
  char *description;
  strcpy(name, "Raghav Rai");
  /* allocate memory dynamically */
  description = malloc( 30 * sizeof(char) );
  if( description == NULL )
  {
    fprintf(stderr, "Error - unable to allocate required memory\n");
  }
  else
  {
    strcpy( description, " Raghav Rai a MVM student in class 11th.");
  }
  /* suppose you want to store bigger description */
  description = realloc( description, 100 * sizeof(char) );
  if( description == NULL )
  {
    fprintf(stderr, "Error - unable to allocate required memory\n");
```

```
  }
  else
  {
    strcat( description, "He is in class 11th");
  }
  printf("Name = %s\n", name );
  printf("Description: %s\n", description );
  /* release memory using free() function */
  free(description);
}
```

Output:

```
Name = Raghav Rai
Description: Raghav Rai a MVM student. He is in class 11th
```

This example without re-allocating extra memory, and strcat() function will give an error due to lack of available memory in description.

malloc()

Allocates a block of memory of specified size

realloc()

Resizes an already allocated memory block

free()

Deallocates previously allocated memory